



Shayan Shuvo Wasif

CSE200

Group Presentation

Shayan

1. Problem & motivation
2. Core idea
3. Construction
4. Representation
5. Complexity

Shuvo

6. Range query
7. Point update
8. C++ implementation
9. Dry run
10. Variants of stored operations

Wasif

11. Lazy propagation
12. Iterative & dynamic trees
13. Persistent tree
14. Comparisons
15. Applications & summary

The recurring problem

Given an array, answer many **range queries** while values may also change.

The recurring problem

Given an array, answer many **range queries** while values may also change.

Example

Array: $A = [1, 4, 5, 9, 10, 12]$

Query: **sum of indices 1 through 4** $= 4 + 5 + 9 + 10 = 28$.

The recurring problem

Given an array, answer many **range queries** while values may also change.

Example

Array: $A = [1, 4, 5, 9, 10, 12]$

Query: **sum of indices 1 through 4** $= 4 + 5 + 9 + 10 = 28$.

Naive approach

- ▶ Scan every element in the range.
- ▶ Worst case: $O(n)$ per query.

Goal

- ▶ Preprocess the array.
- ▶ Answer a query in about $O(\log n)$.
- ▶ Still support updates efficiently.

A segment tree is a **binary tree over contiguous intervals**.

A segment tree is a **binary tree over contiguous intervals**.

- ▶ Root represents the whole array.

A segment tree is a **binary tree over contiguous intervals**.

- ▶ Root represents the whole array.
- ▶ Each internal node represents one interval and splits it into two children.

A segment tree is a **binary tree over contiguous intervals**.

- ▶ Root represents the whole array.
- ▶ Each internal node represents one interval and splits it into two children.
- ▶ Leaves represent individual elements.

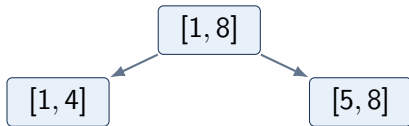
A segment tree is a **binary tree over contiguous intervals**.

- ▶ Root represents the whole array.
- ▶ Each internal node represents one interval and splits it into two children.
- ▶ Leaves represent individual elements.
- ▶ The node value is the result of a chosen operation over its interval.

[1, 8]

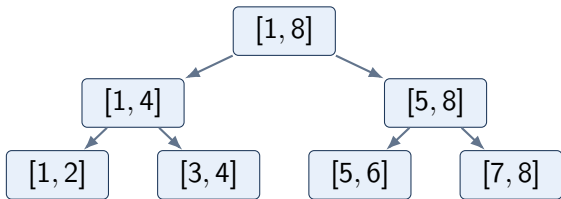
A segment tree is a **binary tree over contiguous intervals**.

- ▶ Root represents the whole array.
- ▶ Each internal node represents one interval and splits it into two children.
- ▶ Leaves represent individual elements.
- ▶ The node value is the result of a chosen operation over its interval.



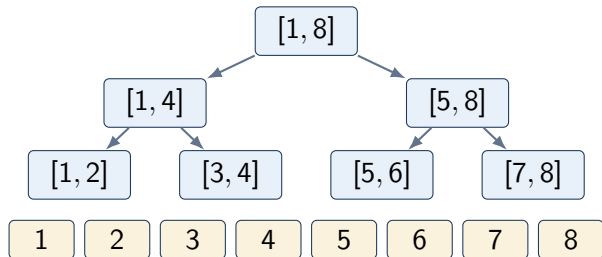
A segment tree is a **binary tree over contiguous intervals**.

- ▶ Root represents the whole array.
- ▶ Each internal node represents one interval and splits it into two children.
- ▶ Leaves represent individual elements.
- ▶ The node value is the result of a chosen operation over its interval.



A segment tree is a **binary tree over contiguous intervals**.

- ▶ Root represents the whole array.
- ▶ Each internal node represents one interval and splits it into two children.
- ▶ Leaves represent individual elements.
- ▶ The node value is the result of a chosen operation over its interval.



Array: [1, 4, 5, 9, 10, 12, 7, 3]

Array: [1, 4, 5, 9, 10, 12, 7, 3]

- 1 Start with [1, 8].

Array: [1, 4, 5, 9, 10, 12, 7, 3]

- ① Start with [1, 8].
- ② Split at $mid = 4$: [1, 4], [5, 8].

Array: [1, 4, 5, 9, 10, 12, 7, 3]

- ① Start with [1, 8].
- ② Split at $mid = 4$: [1, 4], [5, 8].
- ③ Keep splitting until every segment contains one element.

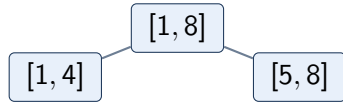
Array: [1, 4, 5, 9, 10, 12, 7, 3]

[1, 8]

- ① Start with [1, 8].
- ② Split at $mid = 4$: [1, 4], [5, 8].
- ③ Keep splitting until every segment contains one element.
- ④ Merge child answers upward.

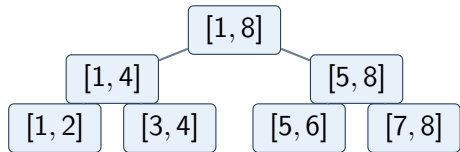
Array: [1, 4, 5, 9, 10, 12, 7, 3]

- 1 Start with [1, 8].
- 2 Split at $mid = 4$: [1, 4], [5, 8].
- 3 Keep splitting until every segment contains one element.
- 4 Merge child answers upward.



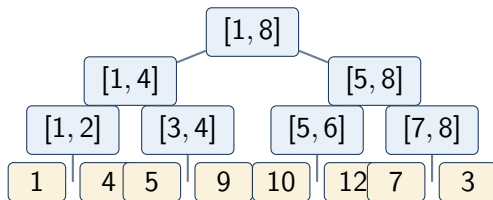
Array: [1, 4, 5, 9, 10, 12, 7, 3]

- 1 Start with [1, 8].
- 2 Split at $mid = 4$: [1, 4], [5, 8].
- 3 Keep splitting until every segment contains one element.
- 4 Merge child answers upward.



Array: [1, 4, 5, 9, 10, 12, 7, 3]

- 1 Start with [1, 8].
- 2 Split at $mid = 4$: [1, 4], [5, 8].
- 3 Keep splitting until every segment contains one element.
- 4 Merge child answers upward.



The operation determines the node meaning

For a sum tree, a node stores the **sum** of its interval. For a minimum tree, it stores the **minimum**; for maximum, the **maximum**; and so on.

The operation determines the node meaning

For a sum tree, a node stores the **sum** of its interval. For a minimum tree, it stores the **minimum**; for maximum, the **maximum**; and so on.

Sum

$$\text{merge}(a, b) = a + b$$

Minimum

$$\text{merge}(a, b) = \min(a, b)$$

Maximum

$$\text{merge}(a, b) = \max(a, b)$$

The operation determines the node meaning

For a sum tree, a node stores the **sum** of its interval. For a minimum tree, it stores the **minimum**; for maximum, the **maximum**; and so on.

Sum

$$\text{merge}(a, b) = a + b$$

Minimum

$$\text{merge}(a, b) = \min(a, b)$$

Maximum

$$\text{merge}(a, b) = \max(a, b)$$

Key requirement

We need a fast **merge/combine** operation that produces the parent answer from its two children.

A segment tree is usually stored in an array rather than explicit node objects.

A segment tree is usually stored in an array rather than explicit node objects.

- ▶ Root index: 1

A segment tree is usually stored in an array rather than explicit node objects.

- ▶ Root index: 1
- ▶ Left child of node i : $2i$

A segment tree is usually stored in an array rather than explicit node objects.

- ▶ Root index: 1
- ▶ Left child of node i : $2i$
- ▶ Right child of node i : $2i + 1$

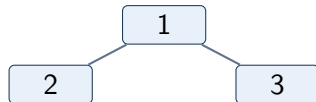
A segment tree is usually stored in an array rather than explicit node objects.

1

- ▶ Root index: 1
- ▶ Left child of node i : $2i$
- ▶ Right child of node i : $2i + 1$
- ▶ Common allocation: about $4n$ positions for a recursive implementation.

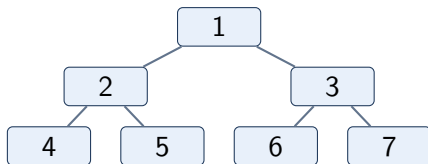
A segment tree is usually stored in an array rather than explicit node objects.

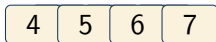
- ▶ Root index: 1
- ▶ Left child of node i : $2i$
- ▶ Right child of node i : $2i + 1$
- ▶ Common allocation: about $4n$ positions for a recursive implementation.



A segment tree is usually stored in an array rather than explicit node objects.

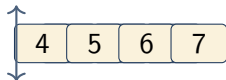
- ▶ Root index: 1
- ▶ Left child of node i : $2i$
- ▶ Right child of node i : $2i + 1$
- ▶ Common allocation: about $4n$ positions for a recursive implementation.





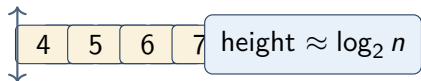
Consequence

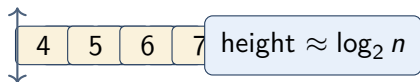
Every root-to-leaf path has logarithmic length, so a point operation follows only about $\log_2 n$ levels.



Consequence

Every root-to-leaf path has logarithmic length, so a point operation follows only about $\log_2 n$ levels.





Consequence

Every root-to-leaf path has logarithmic length, so a point operation follows only about $\log_2 n$ levels.

Operation	Complexity	Why?
Build	$O(n)$	Each node is computed once.
Range query	$O(\log n)$	Visit only a logarithmic number of relevant branches.
Point update	$O(\log n)$	Only one root-to-leaf path changes.
Space	$O(n)$	The tree has a linear number of nodes.

Operation	Complexity	Why?
Build	$O(n)$	Each node is computed once.
Range query	$O(\log n)$	Visit only a logarithmic number of relevant branches.
Point update	$O(\log n)$	Only one root-to-leaf path changes.
Space	$O(n)$	The tree has a linear number of nodes.

The payoff: fast queries *and* fast updates.

At a node representing $[l, r]$, compare it with the query interval $[q_l, q_r]$.

At a node representing $[l, r]$, compare it with the query interval $[q_l, q_r]$.

1. No overlap

$$r < q_l \text{ or } q_r < l$$

Return the identity value.
For sum: 0.

2. Complete overlap

$$q_l \leq l \text{ and } r \leq q_r$$

Return the node's stored answer immediately.

3. Partial overlap

Split at *mid* and query both children.

At a node representing $[l, r]$, compare it with the query interval $[q_l, q_r]$.

1. No overlap

$$r < q_l \text{ or } q_r < l$$

Return the identity value.
For sum: 0.

2. Complete overlap

$$q_l \leq l \text{ and } r \leq q_r$$

Return the node's stored answer immediately.

3. Partial overlap

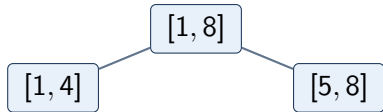
Split at *mid* and query both children.

Why it is fast

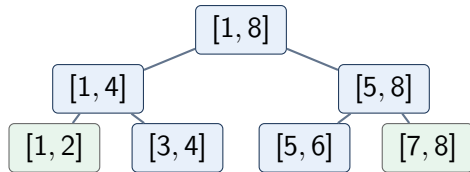
We never visit every leaf. Large fully covered segments are answered in one step.

[1, 8]

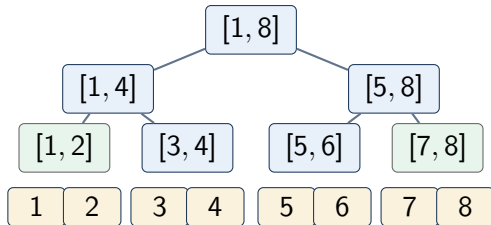
Green nodes are answered directly; other branches are explored only as needed.



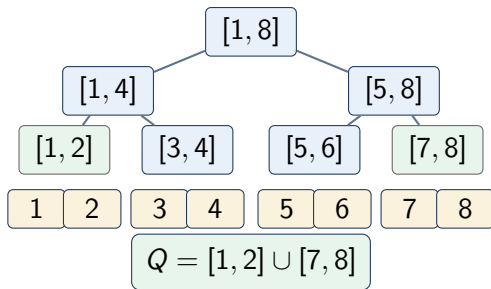
Green nodes are answered directly; other branches are explored only as needed.

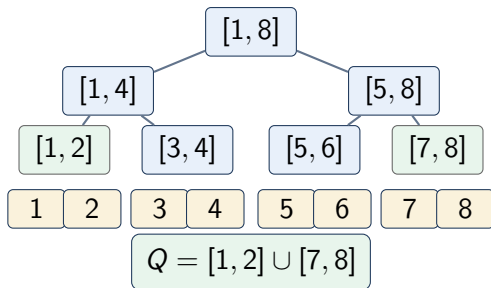


Green nodes are answered directly; other branches are explored only as needed.



Green nodes are answered directly; other branches are explored only as needed.





Green nodes are answered directly; other branches are explored only as needed.

- 1 Start at the root.

- ① Start at the root.
- ② If there is no overlap, return identity.

- ① Start at the root.
- ② If there is no overlap, return identity.
- ③ If there is complete overlap, return the stored value.

- ① Start at the root.
- ② If there is no overlap, return identity.
- ③ If there is complete overlap, return the stored value.
- ④ Otherwise split at *mid*.

- ① Start at the root.
- ② If there is no overlap, return identity.
- ③ If there is complete overlap, return the stored value.
- ④ Otherwise split at *mid*.
- ⑤ Combine the left and right answers.

Sum query

$$\text{query}(ql, qr) = \text{leftAnswer} + \text{rightAnswer}$$

- ① Start at the root.
- ② If there is no overlap, return identity.
- ③ If there is complete overlap, return the stored value.
- ④ Otherwise split at *mid*.
- ⑤ Combine the left and right answers.

Sum query

$$query(ql, qr) = \text{leftAnswer} + \text{rightAnswer}$$

Identity values

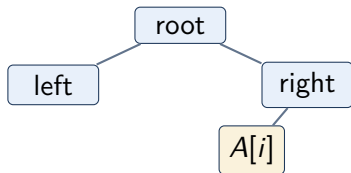
Sum $\rightarrow 0$

Minimum $\rightarrow +\infty$

Maximum $\rightarrow -\infty$

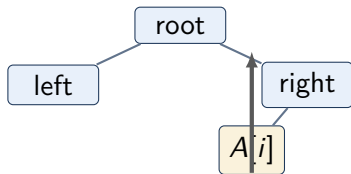
Suppose $A[i]$ changes from v_{old} to v_{new} .

Suppose $A[i]$ changes from v_{old} to v_{new} .



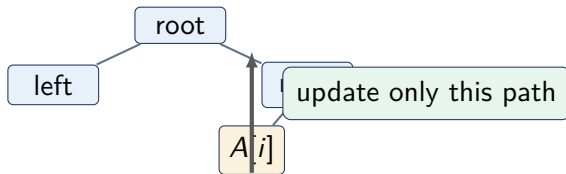
Only $O(\log n)$ ancestors can change, because one element belongs to exactly one node at every tree level.

Suppose $A[i]$ changes from v_{old} to v_{new} .

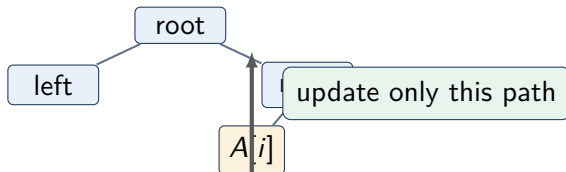


Only $O(\log n)$ ancestors can change, because one element belongs to exactly one node at every tree level.

Suppose $A[i]$ changes from v_{old} to v_{new} .



Suppose $A[i]$ changes from v_{old} to v_{new} .



Only $O(\log n)$ ancestors can change, because one element belongs to exactly one node at every tree level.

```
1 void build(int node, int l, int r) {
2     if (l == r) { tree[node] = a[l]; return; }
3     int mid = (l + r) / 2;
4     build(node * 2, l, mid);
5     build(node * 2 + 1, mid + 1, r);
6     tree[node] = tree[node * 2] + tree[node * 2 + 1];
7 }
8
9 long long query(int node, int l, int r, int ql, int qr) {
10     if (qr < l || r < ql) return 0;
11     if (ql <= l && r <= qr) return tree[node];
12     int mid = (l + r) / 2;
13     return query(node*2, l, mid, ql, qr)
14         + query(node*2+1, mid+1, r, ql, qr);
15 }
```

```
1 void update(int node, int l, int r, int idx, long long val) {
2     if (l == r) { tree[node] = val; return; }
3
4     int mid = (l + r) / 2;
5     if (idx <= mid)
6         update(node*2, l, mid, idx, val);
7     else
8         update(node*2+1, mid+1, r, idx, val);
9
10    tree[node] = tree[node*2] + tree[node*2+1];
11 }
```

```
1 void update(int node, int l, int r, int idx, long long val) {
2     if (l == r) { tree[node] = val; return; }
3
4     int mid = (l + r) / 2;
5     if (idx <= mid)
6         update(node*2, l, mid, idx, val);
7     else
8         update(node*2+1, mid+1, r, idx, val);
9
10    tree[node] = tree[node*2] + tree[node*2+1];
11 }
```

Important invariant

After every update, every internal node still equals the merge of its two children.

Initial array: [2, 1, 3, 4, 5, 2, 6, 1]

Initial array: $[2, 1, 3, 4, 5, 2, 6, 1]$

① Query sum on $[2, 6]$.

Initial array: [2, 1, 3, 4, 5, 2, 6, 1]

- ① Query sum on [2, 6].
- ② Tree decomposes the query into a small set of stored segments.

Initial array: $[2, 1, 3, 4, 5, 2, 6, 1]$

- ① Query sum on $[2, 6]$.
- ② Tree decomposes the query into a small set of stored segments.
- ③ Suppose $A[5]$ changes from $5 \rightarrow 8$.

Initial array: [2, 1, 3, 4, 5, 2, 6, 1]

- ① Query sum on [2, 6].
- ② Tree decomposes the query into a small set of stored segments.
- ③ Suppose $A[5]$ changes from 5 $\rightarrow$ 8.
- ④ Travel from leaf 5 back to the root, recomputing each ancestor.

Initial array: [2, 1, 3, 4, 5, 2, 6, 1]

- ① Query sum on [2, 6].
- ② Tree decomposes the query into a small set of stored segments.
- ③ Suppose $A[5]$ changes from 5 $\rightarrow$ 8.
- ④ Travel from leaf 5 back to the root, recomputing each ancestor.
- ⑤ A future query sees the new value automatically.

Initial array: [2, 1, 3, 4, 5, 2, 6, 1]

- 1 Query sum on [2, 6].
- 2 Tree decomposes the query into a small set of stored segments.
- 3 Suppose $A[5]$ changes from 5 $\rightarrow$ 8.
- 4 Travel from leaf 5 back to the root, recomputing each ancestor.
- 5 A future query sees the new value automatically.

Both query and point update remain $O(\log n)$.

Stored value	Merge operation	Identity
Sum	$a + b$	0
Minimum	$\min(a, b)$	$+\infty$
Maximum	$\max(a, b)$	$-\infty$
GCD	$\gcd(a, b)$	0
AND	$a \& b$	all-bits-one
OR	$a \mid b$	0

Stored value	Merge operation	Identity
Sum	$a + b$	0
Minimum	$\min(a, b)$	$+\infty$
Maximum	$\max(a, b)$	$-\infty$
GCD	$\gcd(a, b)$	0
AND	$a \& b$	all-bits-one
OR	$a b$	0

General pattern

Design each node to store exactly the information needed to merge two child intervals.

Problem

What if an update says: **add 5 to every element in $[l, r]$** ?

Problem

What if an update says: **add 5 to every element in $[l, r]$** ?

- Updating every element individually can take $O(n)$.

Problem

What if an update says: **add 5 to every element in $[l, r]$** ?

- ▶ Updating every element individually can take $O(n)$.
- ▶ Many range updates destroy the advantage of a basic tree.

Problem

What if an update says: **add 5 to every element in $[l, r]$** ?

- ▶ Updating every element individually can take $O(n)$.
- ▶ Many range updates destroy the advantage of a basic tree.
- ▶ We need to postpone work without losing correctness.

Problem

What if an update says: **add 5 to every element in $[l, r]$** ?

- ▶ Updating every element individually can take $O(n)$.
- ▶ Many range updates destroy the advantage of a basic tree.
- ▶ We need to postpone work without losing correctness.

Solution: Lazy propagation

Store a **pending update** at a node and push it to children only when necessary.

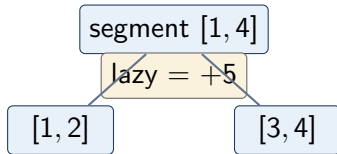
segment [1, 4]

- ▶ Update the covered parent immediately.
- ▶ Record the deferred work in a lazy tag.
- ▶ Push it to children only when recursion needs those children.

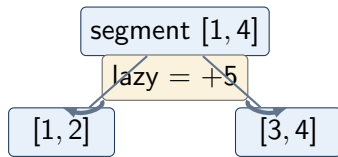
segment [1, 4]

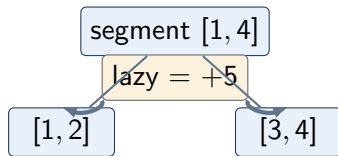
lazy = +5

- ▶ Update the covered parent immediately.
- ▶ Record the deferred work in a lazy tag.
- ▶ Push it to children only when recursion needs those children.

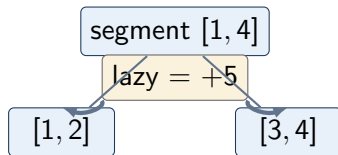


- Update the covered parent immediately.
- Record the deferred work in a lazy tag.
- Push it to children only when recursion needs those children.

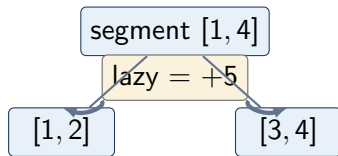




- Update the covered parent immediately.



- Update the covered parent immediately.
- Record the deferred work in a lazy tag.



- Update the covered parent immediately.
- Record the deferred work in a lazy tag.
- Push it to children only when recursion needs those children.

For an update $[u_l, u_r]$ at node $[l, r]$:

For an update $[u_l, u_r]$ at node $[l, r]$:

- 1 Apply any pending lazy update first.

For an update $[u_l, u_r]$ at node $[l, r]$:

- 1 Apply any pending lazy update first.
- 2 If there is no overlap, return.

For an update $[u_l, u_r]$ at node $[l, r]$:

- ➊ Apply any pending lazy update first.
- ➋ If there is no overlap, return.
- ➌ If $[l, r]$ is completely covered, update this node and mark children lazy.

For an update $[u_l, u_r]$ at node $[l, r]$:

- ① Apply any pending lazy update first.
- ② If there is no overlap, return.
- ③ If $[l, r]$ is completely covered, update this node and mark children lazy.
- ④ Otherwise push the lazy tag, recurse to both children, then merge them back.

For an update $[u_l, u_r]$ at node $[l, r]$:

- ① Apply any pending lazy update first.
- ② If there is no overlap, return.
- ③ If $[l, r]$ is completely covered, update this node and mark children lazy.
- ④ Otherwise push the lazy tag, recurse to both children, then merge them back.

Typical result

Range update: $O(\log n)$

Range query: $O(\log n)$

The tree can also be stored bottom-up in a flat array.

The tree can also be stored bottom-up in a flat array.

- ▶ Leaves occupy a contiguous block.

The tree can also be stored bottom-up in a flat array.

- ▶ Leaves occupy a contiguous block.
- ▶ Parents are built upward.

The tree can also be stored bottom-up in a flat array.

- ▶ Leaves occupy a contiguous block.
- ▶ Parents are built upward.
- ▶ Queries often use a while-loop instead of recursion.

The tree can also be stored bottom-up in a flat array.

- ▶ Leaves occupy a contiguous block.
- ▶ Parents are built upward.
- ▶ Queries often use a while-loop instead of recursion.
- ▶ This can reduce recursion overhead and simplify some competitive-programming implementations.

When useful?

- ▶ Point update + range query
- ▶ Large input sizes
- ▶ Cache-friendly implementations

When the index range is huge

Suppose indices may range up to 10^9 or larger, but only a small number of positions are actually touched.

When the index range is huge

Suppose indices may range up to 10^9 or larger, but only a small number of positions are actually touched.

- ▶ Do not allocate a complete tree for every possible index.

When the index range is huge

Suppose indices may range up to 10^9 or larger, but only a small number of positions are actually touched.

- ▶ Do not allocate a complete tree for every possible index.
- ▶ Create nodes only for visited segments.

When the index range is huge

Suppose indices may range up to 10^9 or larger, but only a small number of positions are actually touched.

- ▶ Do not allocate a complete tree for every possible index.
- ▶ Create nodes only for visited segments.
- ▶ Useful for sparse coordinates and online queries.

When the index range is huge

Suppose indices may range up to 10^9 or larger, but only a small number of positions are actually touched.

- ▶ Do not allocate a complete tree for every possible index.
- ▶ Create nodes only for visited segments.
- ▶ Useful for sparse coordinates and online queries.

Trade memory for flexibility: create structure on demand.

New requirement

What if we want to query not only the current array, but also **older versions**?

New requirement

What if we want to query not only the current array, but also **older versions**?

- ▶ A persistent tree keeps previous versions available.

New requirement

What if we want to query not only the current array, but also **older versions**?

- ▶ A persistent tree keeps previous versions available.
- ▶ An update creates only the nodes on the changed path; untouched nodes are shared.

New requirement

What if we want to query not only the current array, but also **older versions**?

- ▶ A persistent tree keeps previous versions available.
- ▶ An update creates only the nodes on the changed path; untouched nodes are shared.
- ▶ This is called **path copying**.

New requirement

What if we want to query not only the current array, but also **older versions**?

- ▶ A persistent tree keeps previous versions available.
- ▶ An update creates only the nodes on the changed path; untouched nodes are shared.
- ▶ This is called **path copying**.

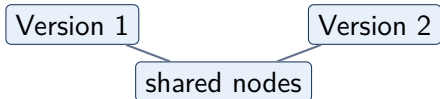
Version 1

Version 2

New requirement

What if we want to query not only the current array, but also **older versions**?

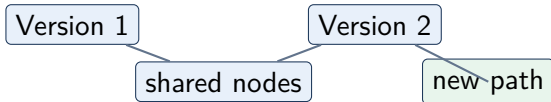
- ▶ A persistent tree keeps previous versions available.
- ▶ An update creates only the nodes on the changed path; untouched nodes are shared.
- ▶ This is called **path copying**.



New requirement

What if we want to query not only the current array, but also **older versions**?

- ▶ A persistent tree keeps previous versions available.
- ▶ An update creates only the nodes on the changed path; untouched nodes are shared.
- ▶ This is called **path copying**.



	Prefix Sum	Fenwick Tree	Segment Tree	Sparse Table
Static range sum	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Point update	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Range minimum	limited	limited	✓	✓
Range update	✗	limited	✓*	✗
Memory	$O(n)$	$O(n)$	$O(n)$	$O(n \log n)$

	Prefix Sum	Fenwick Tree	Segment Tree	Sparse Table
Static range sum	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Point update	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Range minimum	limited	limited	✓	✓
Range update	✗	limited	✓*	✗
Memory	$O(n)$	$O(n)$	$O(n)$	$O(n \log n)$

*With an appropriate lazy-propagation design.

Advantages

- ▶ Dynamic range queries and updates.
- ▶ Flexible node information.
- ▶ Supports complex merges.
- ▶ Extendable with lazy, dynamic, or persistent variants.

Limitations

- ▶ More code than prefix sums or a Fenwick tree.
- ▶ Higher constant factors.
- ▶ More memory than the original array.
- ▶ Correctness depends on choosing the right merge and identity.

Competitive Programming

Range sums, min/max, GCD, frequency and inversion-style problems.

Interval Data

Fast aggregation over changing ranges, scheduling and resource allocation.

Advanced Algorithms

Persistent queries, coordinate-based events and dynamic sparse domains.

Competitive Programming

Range sums, min/max, GCD, frequency and inversion-style problems.

Interval Data

Fast aggregation over changing ranges, scheduling and resource allocation.

Advanced Algorithms

Persistent queries, coordinate-based events and dynamic sparse domains.

A segment tree is most valuable when **the range changes, the data changes, or both.**

Divide

- ▶ Divide the array into intervals.
- ▶ Store a useful summary in every interval.
- ▶ Merge child summaries to answer larger intervals.
- ▶ Use logarithmic height to avoid touching the whole array.

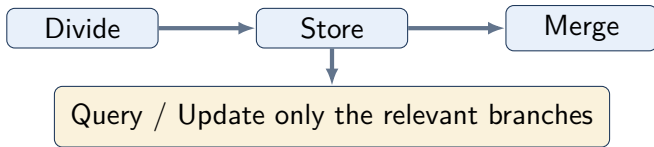
Divide

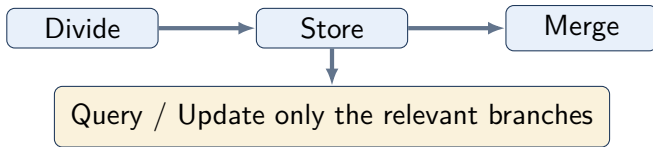
Store

- ▶ Divide the array into intervals.
- ▶ Store a useful summary in every interval.
- ▶ Merge child summaries to answer larger intervals.
- ▶ Use logarithmic height to avoid touching the whole array.

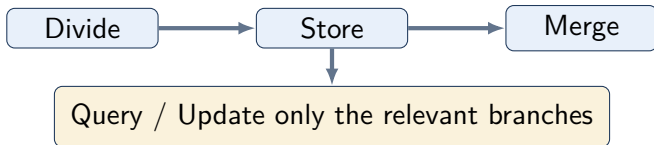


- ▶ Divide the array into intervals.
- ▶ Store a useful summary in every interval.
- ▶ Merge child summaries to answer larger intervals.
- ▶ Use logarithmic height to avoid touching the whole array.

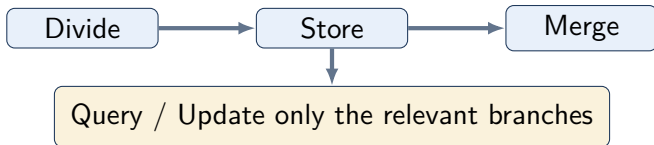




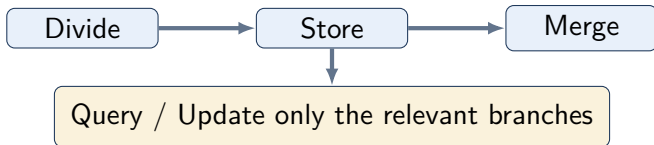
- Divide the array into intervals.



- ▶ Divide the array into intervals.
- ▶ Store a useful summary in every interval.



- ▶ Divide the array into intervals.
- ▶ Store a useful summary in every interval.
- ▶ Merge child summaries to answer larger intervals.



- ▶ Divide the array into intervals.
- ▶ Store a useful summary in every interval.
- ▶ Merge child summaries to answer larger intervals.
- ▶ Use logarithmic height to avoid touching the whole array.

- ▶ A segment tree turns repeated range work into a structured tree problem.

- ▶ A segment tree turns repeated range work into a structured tree problem.
- ▶ Basic build is $O(n)$; point query/update operations are typically $O(\log n)$.

- ▶ A segment tree turns repeated range work into a structured tree problem.
- ▶ Basic build is $O(n)$; point query/update operations are typically $O(\log n)$.
- ▶ The key design pieces are **intervals, merge, and identity**.

- ▶ A segment tree turns repeated range work into a structured tree problem.
- ▶ Basic build is $O(n)$; point query/update operations are typically $O(\log n)$.
- ▶ The key design pieces are **intervals, merge, and identity**.
- ▶ Lazy propagation extends the idea to efficient range updates.

- ▶ A segment tree turns repeated range work into a structured tree problem.
- ▶ Basic build is $O(n)$; point query/update operations are typically $O(\log n)$.
- ▶ The key design pieces are **intervals, merge, and identity**.
- ▶ Lazy propagation extends the idea to efficient range updates.
- ▶ Iterative, dynamic and persistent versions adapt the same idea to different constraints.

- ▶ A segment tree turns repeated range work into a structured tree problem.
- ▶ Basic build is $O(n)$; point query/update operations are typically $O(\log n)$.
- ▶ The key design pieces are **intervals, merge, and identity**.
- ▶ Lazy propagation extends the idea to efficient range updates.
- ▶ Iterative, dynamic and persistent versions adapt the same idea to different constraints.

Thank You

Questions?